

Apache Spark

Aspectos Básicos — Semana 5

Agenda

- **Introducción a Apache Spark** — qué es, historia, comparación con Hadoop, casos de uso, stack
- **Arquitectura de Spark** — Driver, SparkContext/SparkSession, Cluster Manager, Executors, DAG Scheduler
- **RDDs** — definición, propiedades, transformaciones, acciones, particiones y linaje
- **PySpark** — API Python, RDD API, DataFrame API, Spark SQL

SECCIÓN 1

Introducción a Apache Spark

¿Qué es Apache Spark?

- Motor de analítica **unificado** para procesamiento de datos a gran escala
- Creado en **AMPLab (UC Berkeley)** en 2009 por Matei Zaharia
- Innovación clave: **computación en memoria** — evita el I/O repetitivo de disco de MapReduce
- Open-sourced en 2010 → donado a la **Apache Software Foundation** en 2013 → proyecto top-level en 2014
- Versión actual: **Spark 4.x** — soporta batch, streaming, SQL, Machine Learning y grafos en un solo motor

Spark puede ser hasta **100× más rápido** que Hadoop MapReduce en workloads iterativos al mantener los datos en memoria entre operaciones.

(Zaharia et al., 2010)

Apache Spark vs. Hadoop MapReduce

Dimensión	Hadoop MapReduce	Apache Spark
Procesamiento	Solo batch, basado en disco	Batch + Streaming + SQL + ML
Velocidad	Referencia base	Hasta 100x más rápido (iterativo)
Modelo de cómputo	Map → Reduce rígido	DAG flexible de transformaciones
Tolerancia a fallos	Replicación en HDFS	Linaje del RDD (recomputación)
Lenguajes	Principalmente Java	Scala, Python, Java, R, SQL
Streaming	No nativo	Spark Streaming / Structured Streaming

(Zaharia et al., 2016)

Casos de Uso de Apache Spark

- **Procesamiento batch** de grandes volúmenes de datos (ETL masivo)
- **Streaming en tiempo real** — detección de fraude, análisis de IoT, logs de seguridad
- **Machine Learning iterativo** — los algoritmos se benefician del cache en memoria (MLlib)
- **Consultas SQL interactivas** — exploración ad hoc de datos estructurados
- **Procesamiento de grafos** — redes sociales, análisis de influencia (GraphX)

Spark es especialmente ventajoso en algoritmos iterativos (e.g., gradient descent) donde los datos se leen múltiples veces — MapReduce leería desde disco en cada iteración.

El Stack de Spark

Componente	Función
Spark Core	Motor base: RDDs, scheduling, gestión de memoria e I/O
Spark SQL	Datos estructurados, DataFrames/Datasets, optimizador Catalyst
Structured Streaming	Procesamiento de streams como micro-batches o continuo
MLlib	Algoritmos de Machine Learning distribuidos a escala
GraphX	Computación sobre grafos distribuidos

Todos los componentes comparten el mismo motor de ejecución y APIs — no es necesario moverse entre frameworks.

(Apache Software Foundation, 2025a; Chambers & Zaharia, 2018)

SECCIÓN 2

Arquitectura de Spark

Componentes de la Arquitectura

Plano de control

- **Driver Program** — cerebro de la aplicación
- **Cluster Manager** — gestión de recursos del clúster

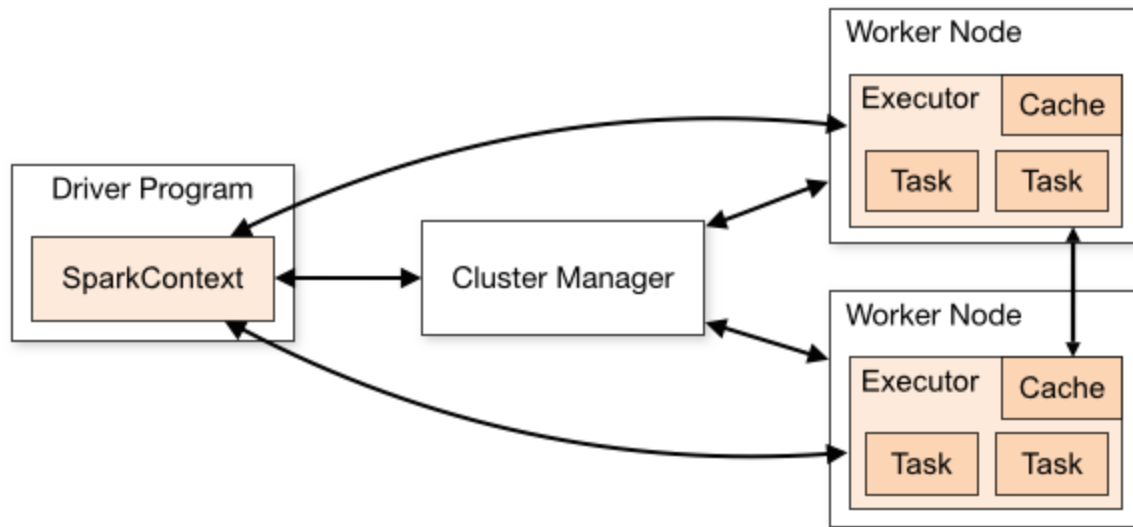
Plano de datos

- **Worker Nodes** — máquinas del clúster
- **Executors** — procesos que ejecutan el trabajo real

Flujo de ejecución

```
Usuario → Driver
Driver → Cluster Manager
          (solicita recursos)
Cluster Manager → Executors
Driver → Executors
          (envía tasks)
Executors → Driver
          (retornan resultados)
```

Esquema Visual



Driver Program

- Proceso JVM que aloja la función `main()` de la aplicación Spark
- **Responsabilidades:**
 - Traduce código Spark de alto nivel en un **DAG** de stages y tasks
 - Coordina la ejecución comunicándose con el Cluster Manager
 - Recolecta resultados finales del clúster al completar cada acción
- Punto crítico: si el Driver falla, la aplicación entera termina
- Contiene el **SparkContext** / **SparkSession** — la conexión al clúster

(Apache Software Foundation, 2025a)

SparkContext y SparkSession

Pre-Spark 2.0

```
from pyspark import SparkContext

sc = SparkContext(
    master="local",
    appName="MiApp"
)
# Solo podía crear RDDs
```

Spark 2.0+ (moderno)

```
from pyspark.sql import SparkSession

spark = SparkSession.builder \
    .appName("MiApp") \
    .master("local[*]") \
    .getOrCreate()

# SparkContext accesible via:
sc = spark.sparkContext
```

`SparkSession` unifica el acceso a RDDs, DataFrames, Datasets y Spark SQL en un único punto de entrada.

Cluster Manager

- Gestiona los **recursos del clúster** y programa las aplicaciones Spark
- Spark es completamente **agnóstico** al Cluster Manager:

Opción	Uso típico
Standalone	Integrado en Spark; ideal para aprendizaje y desarrollo
Apache YARN	Clústeres Hadoop existentes en producción
Apache Mesos	Gestión multi-framework (legacy)
Kubernetes	Despliegues cloud-native modernos

Para esta clase usamos el modo `local[*]`, que no requiere Cluster Manager externo.

(Apache Software Foundation, 2025a)

Executors y Worker Nodes

- **Executor**: proceso JVM que corre en un nodo worker del clúster
 - Ejecuta **tasks** en hilos paralelos (un hilo por core asignado)
 - Almacena particiones de RDDs cacheados en **memoria o disco**
- Cada aplicación Spark tiene sus propios executors → **aislamiento** entre aplicaciones
- Parámetros clave de configuración:

Parámetro	Descripción
<code>spark.executor.cores</code>	Grado de paralelismo por executor
<code>spark.executor.memory</code>	Memoria disponible para cómputo y cache

(Apache Software Foundation, 2025a)

DAG

Del código a la ejecución distribuida

- Un DAG (Grafo Acíclico Dirigido) es el plan de ejecución lógico que estructura cómo se procesan los datos.
- Representa una secuencia de transformaciones (nodos) sin ciclos, optimizando el flujo de trabajo dividiendo tareas en etapas (stages) para ejecutarse en paralelo en un clúster. Se crea automáticamente al usar acciones.

SECCIÓN 3

RDDs — Resilient Distributed Datasets

¿Qué es un RDD?

Resilient Distributed Dataset — abstracción fundamental de datos en Spark

"A distributed memory abstraction that lets programmers perform in-memory computations on large clusters in a fault-tolerant manner."

— Zaharia et al. (2012, p. 2)

- Colección de objetos dividida en **particiones** distribuidas en el clúster
- Definido formalmente en el paper de NSDI 2012 (ganador del **Best Paper Award**)
- Base sobre la que se construyen DataFrames y Datasets en versiones modernas de Spark

Propiedades Fundamentales del RDD

1. **Inmutable** — una vez creado no puede modificarse; las transformaciones producen un nuevo RDD
2. **Distribuido** — dividido en particiones en múltiples nodos del clúster
3. **Resiliente** — se recupera de fallos mediante el **grafo de linaje**, sin necesidad de replicar datos
4. **Evaluación lazy** — las transformaciones no se ejecutan hasta que se invoca una **acción**
5. **Tipado** — `RDD[Int]`, `RDD[String]`, `RDD[(String, Int)]`

La combinación de **inmutabilidad** + **linaje** es lo que hace a los RDDs tolerantes a fallos sin el costo de replicación de HDFS.

(Zaharia et al., 2012)

Transformaciones Narrow

- **No requieren shuffle** — cada partición de salida depende de **una sola** partición de entrada
- Pueden ejecutarse en paralelo sin movimiento de datos por la red

Transformación	Descripción
<code>map(f)</code>	Aplica <code>f</code> a cada elemento — relación 1:1
<code>filter(f)</code>	Conserva solo los elementos donde <code>f</code> retorna <code>True</code>
<code>flatMap(f)</code>	Como <code>map</code> , pero cada elemento puede producir 0 o más salidas
<code>union(rdd2)</code>	Combina dos RDDs sin eliminar duplicados

Narrow transformations son **baratas** — Spark las ejecuta dentro de la misma partición sin coordinar con otros nodos.

Transformaciones Wide (Shuffle)

- Requieren **redistribución de datos entre nodos** → crean nuevos límites de stage en el DAG
- Son más costosas; deben usarse de forma consciente

Transformación	Descripción
<code>reduceByKey(f)</code>	Agrega valores por clave con <code>f</code> — preferir sobre <code>groupByKey</code>
<code>groupByKey()</code>	Agrupar todos los valores por clave — costosa, evitar si hay alternativa
<code>sortByKey()</code>	Ordena el RDD por clave
<code>join(rdd2)</code>	Inner join en RDDs clave-valor
<code>distinct()</code>	Elimina duplicados

Preferir `reduceByKey` sobre `groupByKey` : `reduceByKey` realiza una reducción local por partición *antes* del shuffle, reduciendo los datos transferidos por la red.

(Apache Software Foundation, 2025b; Damji et al., 2020)

Acciones en RDDs

Las **acciones** disparan la ejecución del DAG acumulado y retornan un valor al Driver o escriben en almacenamiento externo.

Acción	Descripción
<code>collect()</code>	Retorna todos los elementos al Driver (<i>precaución con grandes volúmenes</i>)
<code>count()</code>	Número total de elementos en el RDD
<code>take(n)</code>	Retorna los primeros <code>n</code> elementos
<code>first()</code>	Retorna el primer elemento

Acción	Descripción
<code>reduce(f)</code>	Agrega todos los elementos usando la función <code>f</code>
<code>saveAsTextFile(path)</code>	Escribe el RDD a archivos de texto en el sistema de ficheros
<code>foreach(f)</code>	Aplica <code>f</code> a cada elemento (para efectos secundarios, e.g., escritura en BD)

Particiones y Grafo de Linaje

Particiones

- Unidad mínima de paralelismo del RDD
- Un **task** se asigna por partición por stage
- Número configurable:

```
sc.parallelize(data, numPartitions=4)
```

- Más particiones → mayor paralelismo
(hasta el número de cores disponibles)

Grafo de Linaje

- DAG de transformaciones que construyeron el RDD
- Si una partición se pierde por fallo de nodo
→ Spark la **recomputa** desde el linaje
- No necesita replicar datos (diferente a HDFS)
- `rdd.cache()` evita recomputación en accesos repetidos

Referencias

Artículos académicos

Apache Software Foundation. (2025a). *Cluster mode overview* [Documentación].

<https://spark.apache.org/docs/latest/cluster-overview.html>

Apache Software Foundation. (2025b). *RDD programming guide* [Documentación].

<https://spark.apache.org/docs/latest/rdd-programming-guide.html>

Apache Software Foundation. (2025c). *PySpark API reference* [Documentación].

<https://spark.apache.org/docs/latest/api/python/index.html>

Apache Software Foundation. (2025d). *Spark SQL — getting started* [Documentación].

<https://spark.apache.org/docs/latest/sql-getting-started.html>

Referencias (cont.)

Databricks. (2016, julio 14). *A tale of three Apache Spark APIs: RDDs, DataFrames, and Datasets* [Blog post]. <https://www.databricks.com/blog/2016/07/14/a-tale-of-three-apache-spark-apis-rdds-dataframes-and-datasets.html>

Databricks. (2023). *PySpark on Databricks* [Documentación].
<https://docs.databricks.com/aws/en/pyspark/>

Zaharia, M., Chowdhury, M., Franklin, M. J., Shenker, S., & Stoica, I. (2010). *Spark: Cluster computing with working sets*. USENIX Workshop on Hot Topics in Cloud Computing (HotCloud 2010). <https://amplab.cs.berkeley.edu/publication/spark-cluster-computing-with-working-sets-paper/>

Referencias (cont.)

Zaharia, M., Chowdhury, M., Das, T., Dave, A., Ma, J., McCauley, M., Franklin, M. J., Shenker, S., & Stoica, I. (2012). *Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing* [Best Paper]. NSDI 2012.

<https://www.usenix.org/system/files/conference/nsdi12/nsdi12-final138.pdf>

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM*, 59(11), 56–65. <https://doi.org/10.1145/2934664>

¿Preguntas?

hquintan@ulima.edu.pe